CS420 Seminar Papers Summary

October 29, 2026

1 LoRA

Low-Rank Adaptation (LoRA) is a parameter-efficient[1] fine-tuning[2] method. It freezes pre-trained weights and learns low-rank updates for selected weight matrices. These notes follow Hu et al., *LoRA: Low-Rank Adaptation of Large Language Models*.

1.1 Introduction

1.1.1 The problem

Full fine-tuning updates all model parameters. For a large model such as GPT-3 (175 billion parameters), this makes training expensive and requires storing a separate full-size checkpoint for each task. Other parameter-efficient methods reduce task-specific storage, but some add inference latency or use part of the available context window.

1.1.2 The solution

Motivated by evidence that model adaptation can take place in a low-dimensional subspace, the LoRA authors hypothesize that the required weight updates have low intrinsic rank.[3] LoRA therefore represents the update to selected weight matrices with two trainable, low-rank matrices while keeping the original weights fixed.

This gives LoRA several practical advantages:

- A single pre-trained model can be shared across tasks; each task needs only its own small LoRA matrices.
- Training uses fewer trainable parameters and less memory for gradients and optimizer states than full fine-tuning.
- The update can be merged into the original weights for inference, so it adds no inference latency.[4]
- LoRA can be combined with other adaptation methods.

1.2 Formulation

1.2.1 Problem statement

Consider a pre-trained autoregressive language model[5] $P_\Phi(y \mid x)$ with parameters Φ_0. For a downstream task, let $\mathcal{Z} = \{(x_i, y_i)\}_{i=1}^{N}$ be context-target pairs, where x_i and y_i are token

[1] Parameter-efficient methods train relatively few task-specific parameters rather than updating every pre-trained model parameter.

[2] Fine-tuning adapts a pre-trained model to a downstream task using task-specific data.

[3] Here, low intrinsic rank means that the task-specific change to a weight matrix can be represented effectively by a product of two matrices with a small shared dimension.

[4] Inference latency is the time a model takes to produce an output after receiving an input.

[5] An autoregressive language model predicts each next token conditioned on the input and the tokens generated so far.

sequences. Full fine-tuning updates Φ_0 to maximize the conditional log-likelihood:

$$\max_{\Phi} \sum_{(x,y)\in\mathcal{Z}} \sum_{t=1}^{|y|} \log P_{\Phi}(y_t \mid x, y_{<t}).$$

This approach learns a full-size parameter change $\Delta\Phi$ for every task. LoRA instead learns a much smaller set of parameters Θ, which defines updates $\Delta\Phi(\Theta)$ to selected weights while the pre-trained parameters stay frozen. The objective becomes

$$\max_{\Theta} \sum_{(x,y)\in\mathcal{Z}} \sum_{t=1}^{|y|} \log P_{\Phi_0+\Delta\Phi(\Theta)}(y_t \mid x, y_{<t}), \qquad |\Theta| \ll |\Phi_0|.$$

1.2.2 Method

For a pre-trained weight matrix $W_0 \in \mathbb{R}^{d\times k}$, LoRA constrains its update to $\Delta W = BA$, where $B \in \mathbb{R}^{d\times r}$, $A \in \mathbb{R}^{r\times k}$, and $r \ll \min(d, k)$. With input $x \in \mathbb{R}^k$, the adapted layer computes

$$h = W_0 x + \frac{\alpha}{r} BAx,$$

where α is a scaling hyperparameter. The factor α/r reduces the need to retune other hyperparameters when the rank r changes. Only A and B are trained; W_0 remains frozen.

For this matrix, LoRA trains $r(d + k)$ parameters instead of the dk weights that full fine-tuning would update. This reduces the task-specific checkpoint size and the gradient and optimizer-state memory for the adapted matrix. The base weights must still be stored and used.

1.2.3 Initialization and deployment

The original LoRA method initializes A with random Gaussian values and B with zeros. Consequently, $BA = 0$ at the start of training, so the adapted layer initially computes exactly $W_0 x$. Training then changes A and B while leaving W_0 fixed.

After training, the update can be merged into the weight matrix:

$$W_{\text{merged}} = W_0 + \frac{\alpha}{r} BA.$$

Inference with $W_{\text{merged}} x$ uses the same matrix multiplication as the original layer and incurs no additional latency from a separate LoRA path. For multiple tasks, one can retain the shared base model and store a small adapter for each task, merging the chosen adapter when needed. Deploying an adapter separately still requires access to its matching base model.